

# HackNEX — Database Design Document

**Document Version:** 1.0

**Project:** HackNEX

**Organizer:** NEXUS Club, Karunya Institute of Technology and Sciences

**Event Dates:** October 7–9, 2026

**Registration Period:** September 8–October 1, 2026

**Expected Participants:** 1,500+

**Team Size:** Exactly 4

**Registration Fee:** ₹600 per team

**Database:** PostgreSQL

**ORM:** Prisma

**Database Hosting:** Neon

---

## 1. Purpose

This document defines the complete database architecture for the HackNEX platform.

It specifies:

- Database entities
  - Tables
  - Columns
  - Data types
  - Primary keys
  - Foreign keys
  - Unique constraints
  - Indexes
  - Enumerations
  - Relationships
  - Registration state management
  - Payment state management
  - Authentication data
  - Admin data
  - Content management
  - Media management
  - Audit logging
  - Prisma implementation
-

## 2. Database Design Principles

The database will follow these principles:

### 2.1 Relational integrity

Foreign keys will ensure that related records cannot become orphaned.

### 2.2 Server-side enforcement

Important business rules will be enforced at the database/backend level rather than relying only on frontend validation.

### 2.3 Atomic registration

Creating a team and its four participants should occur within a database transaction.

### 2.4 Payment integrity

Payment records must be traceable to a registration and transaction.

### 2.5 Auditability

Important administrative operations should be recorded.

### 2.6 Scalability

The schema should comfortably support:

5,000+ participants

even though the initial expectation is approximately 1,500+.

---

## 3. Database Technology

### Primary Database

PostgreSQL

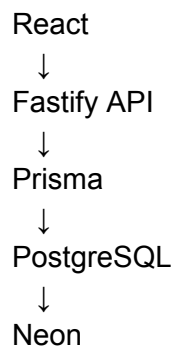
### ORM

Prisma

# Hosting

## Neon PostgreSQL

Architecture:



## 4. Complete Entity List

The database will contain the following major entities.

### Authentication

- User
- Session
- Account
- Verification

### Hackathon Registration

- Team
- Participant
- Registration
- Payment

### Content Management

- Domain
- ScheduleItem
- Prize
- Sponsor
- FAQ
- Announcement

### Media

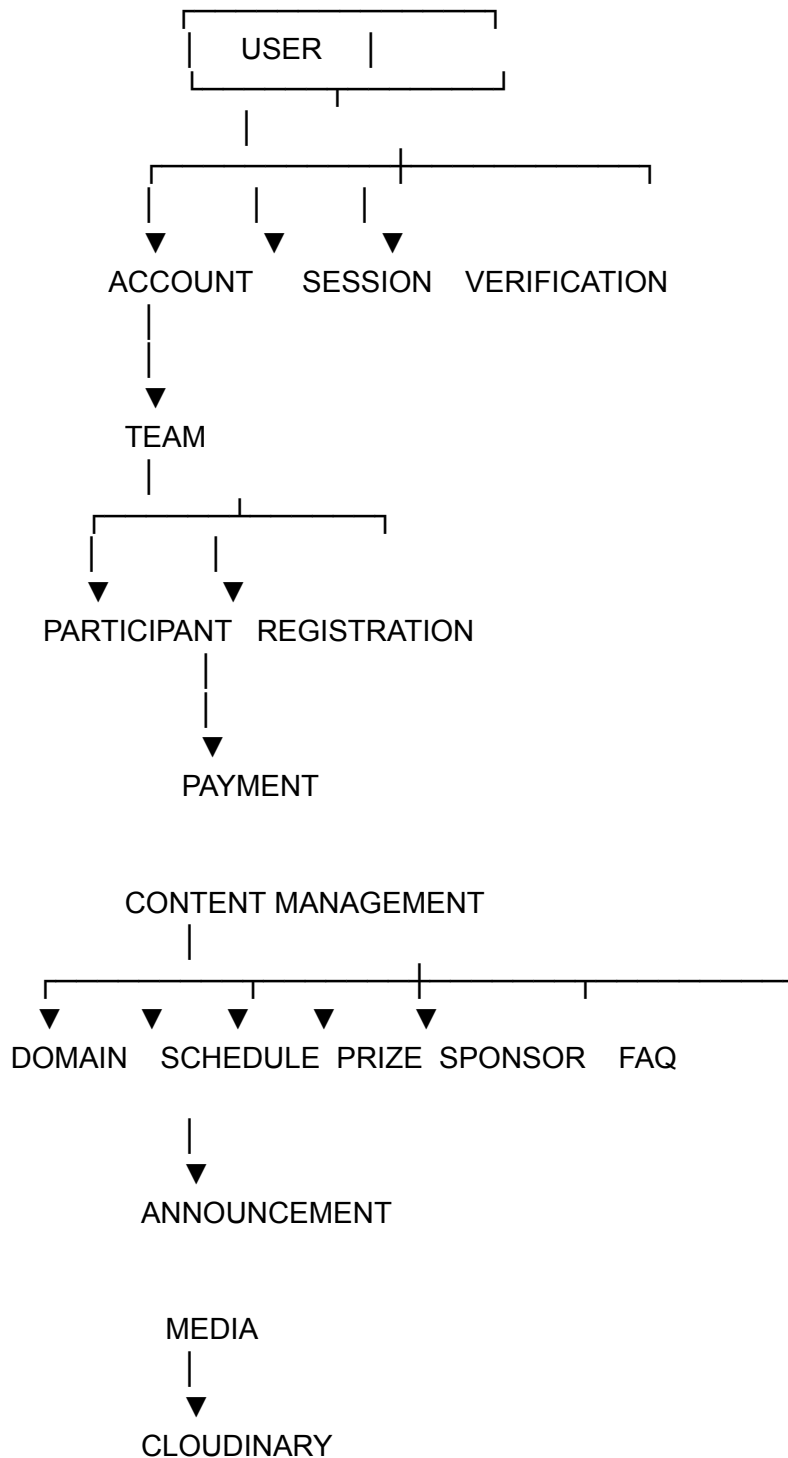
- Media

### Administration

- Admin

## 5. Entity Relationship Diagram

High-level ER diagram:





## 6. Authentication Tables

The authentication system will be based around a user identity and sessions.

---

## 7. User Table

The **User** table represents a registered HackNEX account.

**Table: users**

| Column        | PostgreSQL Type | Constraints      |
|---------------|-----------------|------------------|
| id            | UUID            | PK               |
| name          | VARCHAR(150)    | NOT NULL         |
| email         | VARCHAR(255)    | NOT NULL, UNIQUE |
| emailVerified | BOOLEAN         | NOT NULL         |
| createdAt     | TIMESTAMP       | NOT NULL         |
| updatedAt     | TIMESTAMP       | NOT NULL         |

### Purpose

Stores authentication identities.

---

## 8. Session Table

Stores authenticated sessions.

**Table: sessions**

| Column    | Type      | Constraints |
|-----------|-----------|-------------|
| id        | UUID      | PK          |
| userId    | UUID      | FK          |
| token     | TEXT      | UNIQUE      |
| expiresAt | TIMESTAMP | NOT NULL    |
| createdAt | TIMESTAMP | NOT NULL    |
| updatedAt | TIMESTAMP | NOT NULL    |

Relationship:

User 1 ————— N Session

---

## 9. Account Table

Used by the authentication layer to store authentication provider/account information.

**Table: accounts**

| Column                | Type      | Constraints |
|-----------------------|-----------|-------------|
| id                    | UUID      | PK          |
| userId                | UUID      | FK          |
| accountId             | VARCHAR   | NOT NULL    |
| providerId            | VARCHAR   | NOT NULL    |
| accessToken           | TEXT      | NULL        |
| refreshToken          | TEXT      | NULL        |
| accessTokenExpiresAt  | TIMESTAMP | NULL        |
| refreshTokenExpiresAt | TIMESTAMP | NULL        |
| scope                 | TEXT      | NULL        |
| password              | TEXT      | NULL        |

|           |           |          |
|-----------|-----------|----------|
| createdAt | TIMESTAMP | NOT NULL |
| updatedAt | TIMESTAMP | NOT NULL |

Since HackNEX is using email/password authentication initially, social-login fields may remain unused.

## 10. Verification Table

Used for email verification and password-reset operations.

**Table: verifications**

| Column     | Type         | Constraint<br>s |
|------------|--------------|-----------------|
| id         | UUID         | PK              |
| identifier | VARCHAR(255) | NOT NULL        |
| value      | TEXT         | NOT NULL        |
| expiresAt  | TIMESTAMP    | NOT NULL        |
| createdAt  | TIMESTAMP    | NOT NULL        |
| updatedAt  | TIMESTAMP    | NOT NULL        |

Possible identifiers:

email verification  
password reset

## 11. Team Table

The team is the central entity in HackNEX registration.

**Table: teams**

| Column | Type | Constraints |
|--------|------|-------------|
| id     | UUID | PK          |

|               |              |                  |
|---------------|--------------|------------------|
| teamName      | VARCHAR(100) | NOT NULL, UNIQUE |
| captainUserId | UUID         | FK, UNIQUE       |
| status        | TeamStatus   | NOT NULL         |
| createdAt     | TIMESTAMP    | NOT NULL         |
| updatedAt     | TIMESTAMP    | NOT NULL         |

---

## 12. Team Status

DRAFT  
READY\_FOR\_PAYMENT  
PAYMENT\_PENDING  
PAYMENT\_VERIFIED  
CONFIRMED  
CANCELLED

Recommended meaning:

### **DRAFT**

Team registration is being completed.

### **READY\_FOR\_PAYMENT**

All four participants have been entered and validated.

### **PAYMENT\_PENDING**

Payment has been initiated.

### **PAYMENT\_VERIFIED**

Payment has been successfully verified.

### **CONFIRMED**

Registration is completely successful.

### **CANCELLED**

Registration has been cancelled.

---



## 13. Participant Table

This table stores all four members.

**Table:** **participants**

| Column         | Type           | Constraint<br>s |
|----------------|----------------|-----------------|
| id             | UUID           | PK              |
| teamId         | UUID           | FK              |
| name           | VARCHAR(150)   | NOT NULL        |
| email          | VARCHAR(255)   | NOT NULL        |
| phone          | VARCHAR(20)    | NOT NULL        |
| college        | VARCHAR(200)   | NOT NULL        |
| department     | VARCHAR(150)   | NOT NULL        |
| yearOfStudy    | VARCHAR(50)    | NOT NULL        |
| linkedinUrl    | VARCHAR(500)   | NULL            |
| foodPreference | FoodPreference | NOT NULL        |
| isCaptain      | BOOLEAN        | NOT NULL        |
| createdAt      | TIMESTAMP      | NOT NULL        |
| updatedAt      | TIMESTAMP      | NOT NULL        |

---

## 14. Participant Uniqueness

The system must prevent a participant from registering in multiple teams.

Primary candidate:

email UNIQUE

Therefore:

Participant.email



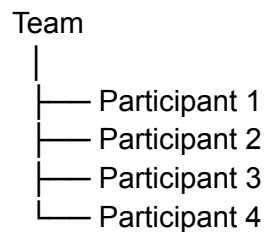
UNIQUE

This means a participant cannot appear in two teams using the same email.

Phone uniqueness can be considered separately, but email is the primary identity constraint.

---

## 15. Participant-Team Relationship



Database relationship:

Team 1 ————— N Participant

The application must ensure:

Maximum = 4 participants.

For a confirmed team:

Exactly 4 participants.

---

## 16. Captain Relationship

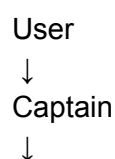
The captain exists in the `participants` table:

`isCaptain = true`

and is also connected to:

`Team.captainUserId`

This creates two useful relationships:



Team

↓

Participant

The backend must ensure that the captain participant belongs to the same team.

---

## 17. Food Preference Enum

```
enum FoodPreference {  
    VEG  
    NON_VEG  
}
```

The captain selects the food preference individually for:

Captain

Member 2

Member 3

Member 4

---

## 18. Year of Study

The current requirement is:

Year of Study

Possible values may be:

1

2

3

4

5

Other

I recommend storing this as an enum only if the organizers have finalized the exact allowed years.

Otherwise:

VARCHAR(50)

is more flexible.

For V1, I recommend:

yearOfStudy VARCHAR(50)

---

## 19. Registration Table

The **Registration** entity represents the actual HackNEX registration transaction.

**Table: registrations**

| Column         | Type               | Constraints |
|----------------|--------------------|-------------|
| id             | UUID               | PK          |
| registrationId | VARCHAR(30)        | UNIQUE      |
| teamId         | UUID               | FK, UNIQUE  |
| status         | RegistrationStatus | NOT NULL    |
| submittedAt    | TIMESTAMP          | NULL        |
| confirmedAt    | TIMESTAMP          | NULL        |
| createdAt      | TIMESTAMP          | NOT NULL    |
| updatedAt      | TIMESTAMP          | NOT NULL    |

---

## 20. Registration Status

DRAFT  
READY\_FOR\_PAYMENT  
PAYMENT\_PENDING  
PAYMENT\_VERIFIED  
CONFIRMED  
CANCELLED

The registration state is separate from the payment state.

This distinction is important.

---

## 21. Why Registration and Payment Need Separate States

Consider:

Registration = PAYMENT\_PENDING  
Payment = PROCESSING

or:

Registration = PAYMENT\_VERIFIED  
Payment = VERIFIED

The two concepts should not be merged into one database field.

---

## 22. Registration ID

Every successful registration receives a unique identifier.

Example:

HNX-2026-000001

Database:

registrationId UNIQUE

---

## 23. Payment Table

Table: **payments**

| Column         | Type        | Constraints |
|----------------|-------------|-------------|
| id             | UUID        | PK          |
| registrationId | UUID        | FK          |
| amount         | INTEGER     | NOT NULL    |
| currency       | VARCHAR(3)  | NOT NULL    |
| provider       | VARCHAR(50) | NOT NULL    |

|                   |               |              |
|-------------------|---------------|--------------|
| transactionId     | VARCHAR(150)  | UNIQUE, NULL |
| status            | PaymentStatus | NOT NULL     |
| providerReference | VARCHAR(255)  | NULL         |
| verifiedAt        | TIMESTAMP     | NULL         |
| createdAt         | TIMESTAMP     | NOT NULL     |
| updatedAt         | TIMESTAMP     | NOT NULL     |

---

## 24. Payment Amount

The official registration fee is:

₹600 per team

Store:

amount = 600  
currency = INR

The backend determines this amount.

The frontend must not be trusted to submit the payment amount.

---

## 25. Payment Status

PENDING  
INITIATED  
PROCESSING  
SUCCESS  
VERIFIED  
FAILED  
EXPIRED  
CANCELLED

---

## 26. Payment Provider

Current provider:

KARUNYA

Example:

provider = "KARUNYA"

The exact API implementation remains dependent on Karunya's payment system.

---

## 27. Payment Relationship

Registration



Payment

Recommended:

Registration 1 ————— N Payment

Why 1 :N?

Because a team might have:

Payment Attempt 1 → Failed

Payment Attempt 2 → Failed

Payment Attempt 3 → Successful

Only the verified successful payment completes the registration.

---

## 28. Payment Transaction ID

The payment provider's transaction identifier must be stored.

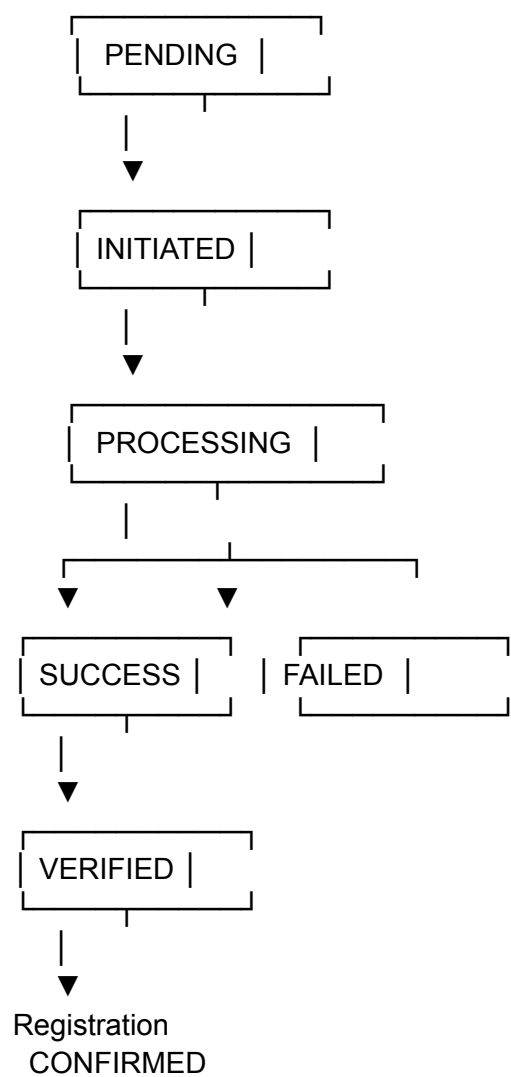
Example:

KARUNYA-TXN-839472

This should have a unique constraint.

---

## 29. Payment State Machine



## 30. Payment Verification Rule

The system must never transition:

Registration → CONFIRMED

until:

Payment → VERIFIED

## 31. Domain Table



Hackathon domains are stored dynamically.

**Table: domains**

| Column       | Type         | Constraint<br>s |
|--------------|--------------|-----------------|
| id           | UUID         | PK              |
| name         | VARCHAR(150) | NOT NULL        |
| slug         | VARCHAR(150) | UNIQUE          |
| description  | TEXT         | NOT NULL        |
| icon         | VARCHAR(255) | NULL            |
| displayOrder | INTEGER      | NOT NULL        |
| isActive     | BOOLEAN      | NOT NULL        |
| createdAt    | TIMESTAMP    | NOT NULL        |
| updatedAt    | TIMESTAMP    | NOT NULL        |

---

## 32. Schedule Table

**Table: schedule\_items**

| Column       | Type         | Constraint<br>s |
|--------------|--------------|-----------------|
| id           | UUID         | PK              |
| title        | VARCHAR(200) | NOT NULL        |
| description  | TEXT         | NULL            |
| startTime    | TIMESTAMP    | NOT NULL        |
| endTime      | TIMESTAMP    | NULL            |
| location     | VARCHAR(200) | NULL            |
| day          | VARCHAR(50)  | NOT NULL        |
| displayOrder | INTEGER      | NOT NULL        |
| isActive     | BOOLEAN      | NOT NULL        |

|           |           |          |
|-----------|-----------|----------|
| createdAt | TIMESTAMP | NOT NULL |
| updatedAt | TIMESTAMP | NOT NULL |

This allows the frontend to render the schedule as a timeline.

---

## 33. Prize Table

Table: **prizes**

| Column       | Type         | Constraint<br>s |
|--------------|--------------|-----------------|
| id           | UUID         | PK              |
| title        | VARCHAR(150) | NOT NULL        |
| description  | TEXT         | NULL            |
| amount       | INTEGER      | NULL            |
| position     | INTEGER      | NULL            |
| displayOrder | INTEGER      | NOT NULL        |
| isActive     | BOOLEAN      | NOT NULL        |
| createdAt    | TIMESTAMP    | NOT NULL        |
| updatedAt    | TIMESTAMP    | NOT NULL        |

---

## 34. Sponsor Table

Table: **sponsors**

| Column  | Type         | Constraint<br>s |
|---------|--------------|-----------------|
| id      | UUID         | PK              |
| name    | VARCHAR(200) | NOT NULL        |
| logoUrl | TEXT         | NULL            |

|              |              |          |
|--------------|--------------|----------|
| websiteUrl   | TEXT         | NULL     |
| tier         | VARCHAR(100) | NULL     |
| displayOrder | INTEGER      | NOT NULL |
| isActive     | BOOLEAN      | NOT NULL |
| createdAt    | TIMESTAMP    | NOT NULL |
| updatedAt    | TIMESTAMP    | NOT NULL |

Sponsor logos can be hosted through Cloudinary.

---

## 35. FAQ Table

Table: **faqs**

| Column       | Type         | Constraint<br>s |
|--------------|--------------|-----------------|
| id           | UUID         | PK              |
| question     | VARCHAR(500) | NOT NULL        |
| answer       | TEXT         | NOT NULL        |
| displayOrder | INTEGER      | NOT NULL        |
| isActive     | BOOLEAN      | NOT NULL        |
| createdAt    | TIMESTAMP    | NOT NULL        |
| updatedAt    | TIMESTAMP    | NOT NULL        |

---

## 36. Announcement Table

Table: **announcements**

| Column | Type | Constraint<br>s |
|--------|------|-----------------|
| id     | UUID | PK              |

|             |              |          |
|-------------|--------------|----------|
| title       | VARCHAR(200) | NOT NULL |
| content     | TEXT         | NOT NULL |
| publishedAt | TIMESTAMP    | NULL     |
| isPublished | BOOLEAN      | NOT NULL |
| createdAt   | TIMESTAMP    | NOT NULL |
| updatedAt   | TIMESTAMP    | NOT NULL |

Announcements can later be used to trigger email notifications.

---

## 37. Media Table

The Media table stores Cloudinary references.

**Table:** **media**

| Column       | Type          | Constraint<br>s |
|--------------|---------------|-----------------|
| id           | UUID          | PK              |
| publicId     | VARCHAR(255)  | UNIQUE          |
| secureUrl    | TEXT          | NOT NULL        |
| resourceType | VARCHAR(50)   | NOT NULL        |
| category     | MediaCategory | NOT NULL        |
| title        | VARCHAR(200)  | NULL            |
| altText      | VARCHAR(500)  | NULL            |
| displayOrder | INTEGER       | NOT NULL        |
| isActive     | BOOLEAN       | NOT NULL        |
| createdAt    | TIMESTAMP     | NOT NULL        |
| updatedAt    | TIMESTAMP     | NOT NULL        |

---

## 38. Media Category Enum

HERO  
ABOUT  
VENUE  
SPONSOR  
ORGANIZER  
OTHER

Initial website requirement:

Approximately 10–20 images.

Architecture capacity:

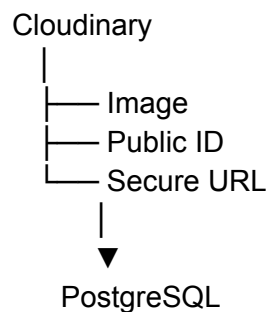
500+ images.

---

## 39. Cloudinary Relationship

The database does **not** store the actual image binary.

Instead:



This keeps PostgreSQL lightweight.

---

## 40. Admin Table

Because there are approximately 15 administrators, an explicit admin representation is useful.

**Table:** **admins**

| Column    | Type      | Constraints |
|-----------|-----------|-------------|
| id        | UUID      | PK          |
| userId    | UUID      | FK, UNIQUE  |
| isActive  | BOOLEAN   | NOT NULL    |
| createdAt | TIMESTAMP | NOT NULL    |
| updatedAt | TIMESTAMP | NOT NULL    |

Relationship:

User 1 ——— 0/1 Admin

---

## 41. Admin Role

Since all administrators currently have equal privileges:

ADMIN

No complex role hierarchy is required for V1.

If future requirements introduce:

SUPER\_ADMIN  
FINANCE\_ADMIN  
CONTENT\_ADMIN

a role system can be added later.

---

## 42. Audit Log Table

Table: **audit\_logs**

| Column  | Type         | Constraint<br>s |
|---------|--------------|-----------------|
| id      | UUID         | PK              |
| adminId | UUID         | FK              |
| action  | VARCHAR(100) | NOT NULL        |

|            |              |          |
|------------|--------------|----------|
| entityType | VARCHAR(100) | NOT NULL |
| entityId   | UUID         | NULL     |
| metadata   | JSONB        | NULL     |
| createdAt  | TIMESTAMP    | NOT NULL |

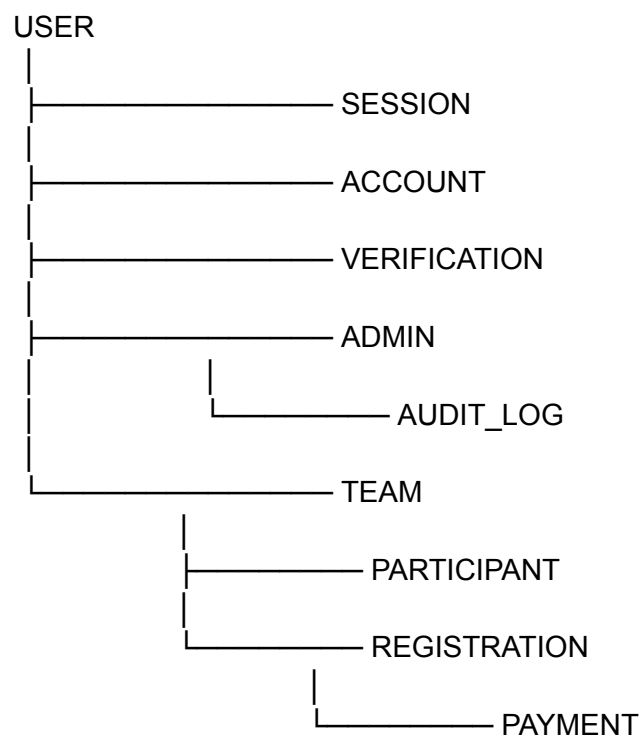
---

## 43. Audit Log Examples

ADMIN\_LOGIN  
 TEAM\_VIEWED  
 TEAM\_UPDATED  
 PARTICIPANT\_UPDATED  
 PAYMENT\_VIEWED  
 MEDIA\_UPLOADED  
 MEDIA\_DELETED  
 FAQ\_CREATED  
 FAQ\_UPDATED  
 SCHEDULE\_UPDATED  
 ANNOUNCEMENT\_PUBLISHED

---

## 44. Complete Database Relationship Map



DOMAIN  
SCHEDULE\_ITEM  
PRIZE  
SPONSOR  
FAQ  
ANNOUNCEMENT  
MEDIA

---

## 45. Primary Keys

All primary keys should use UUIDs.

Example:

550e8400-e29b-41d4-a716-446655440000

Benefits:

- Globally unique
- Harder to guess
- Suitable for distributed systems
- Avoids sequential ID exposure

The human-readable registration ID remains separate.

---

## 46. Foreign Keys

Major foreign keys:

Session.userId  
Account.userId  
Team.captainUserId  
Participant.teamId  
Registration.teamId  
Payment.registrationId  
Admin.userId  
AuditLog.adminId

---



## 47. Unique Constraints

Important unique constraints:

User.email  
Session.token  
Team.teamName  
Team.captainUserId  
Participant.email  
Registration.registrationId  
Registration.teamId  
Payment.transactionId  
Media.publicId  
Domain.slug  
Admin.userId

---

## 48. Indexes

Recommended indexes:

users(email)

participants(email)  
participants(phone)  
participants(college)  
participants(department)  
participants(teamId)

teams(teamName)  
teams(captainUserId)  
teams(status)

registrations(registrationId)  
registrations(status)  
registrations(teamId)

payments(transactionId)  
payments(status)  
payments(registrationId)

schedule\_items(startTime)  
schedule\_items(displayOrder)

media(category)

media(displayOrder)

audit\_logs(adminId)

audit\_logs(entityType)

audit\_logs(createdAt)

---

## 49. Database-Level Team Size Enforcement

PostgreSQL does not conveniently enforce:

"A team must have exactly four rows"

using a simple **CHECK** constraint.

Therefore this business rule will be enforced through:

### Backend transaction logic

and:

### Registration state validation.

For example:

```
if participantCount !== 4:  
    reject registration
```

A database trigger could enforce this, but I **do not recommend one initially** because it adds complexity and makes Prisma-based development harder to maintain.

---

## 50. Registration Transaction

When submitting a completed registration:

BEGIN

Validate Captain

Validate Member 2

Validate Member 3

Validate Member 4

Check participant uniqueness

Check team name uniqueness

Create Team

Create 4 Participants

Create Registration

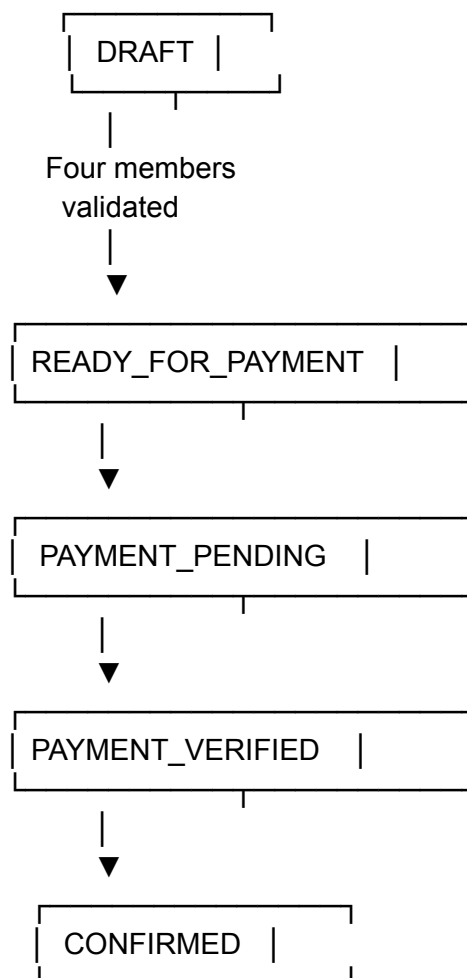
COMMIT

If any operation fails:

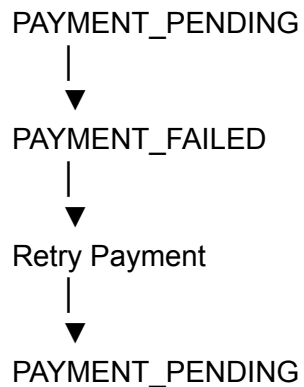
ROLLBACK

---

## 51. Registration State Model



## 52. Failed Registration Flow



The team should not need to re-enter all participant information simply because payment failed.

---

## 53. Confirmation Rule

Final confirmation:

IF

team participant count = 4

AND

registration valid = TRUE

AND

payment status = VERIFIED

THEN

registration status = CONFIRMED

---

## 54. Registration Completion

After confirmation:

Registration



Generate Registration ID



Save confirmation timestamp



Send confirmation email

Example:

HNX-2026-000001

---

## 55. Database Transaction for Payment Confirmation

Payment verification should also be transactional.

BEGIN

Update Payment → VERIFIED

Update Registration → PAYMENT\_VERIFIED

Update Team → CONFIRMED

Generate/assign registration confirmation

COMMIT

If any step fails:

ROLLBACK

---

## 56. Duplicate Payment Protection

Suppose Karunya sends the same transaction twice.

The database checks:

transactionId UNIQUE

The second request should be safely ignored or treated as an already-processed transaction.

---

## 57. Duplicate Participant Protection

Suppose:

infant@example.com

already belongs to Team A.

Team B attempts to register the same email.

Database:

UNIQUE(email)

Result:

Registration rejected.

The frontend can display:

"This participant is already associated with a HackNEX team."

---

## 58. Duplicate Team Protection

Suppose:

TeamName = Code Titans

already exists.

Another team attempts:

code titans

The backend should normalize team names before comparison.

Recommended normalized representation:

lowercase

trim whitespace

collapse repeated spaces

Then check uniqueness.

---

## 59. Data Normalization

Participant data should be normalized before storage.

Example:

" Infant Bosco "

becomes:

"Infant Bosco"

Email:

INFANT@EXAMPLE.COM

becomes:

infant@example.com

This reduces duplicate-data problems.

---

## 60. Deletion Strategy

Critical registration records should generally **not be hard-deleted**.

For example, instead of deleting a confirmed team:

DELETE Team

use:

status = CANCELLED

This preserves historical records.

---

## 61. Content Deletion

For content such as:

- FAQ
- Schedule
- Sponsor
- Domain

soft deletion using:

isActive = false

is recommended.

---

## 62. Media Deletion

Media deletion has two steps:

Database



Remove media record



Cloudinary



Delete asset

The Cloudinary deletion should be handled carefully to prevent orphaned media.

---

## 63. JSONB Usage

JSONB should be used sparingly.

Recommended use:

AuditLog.metadata

Example:

```
{
  "previousStatus": "PENDING",
  "newStatus": "VERIFIED",
  "transactionId": "..."
}
```

Core business data should remain in normal relational columns.

---

## 64. Timestamps

All major tables should contain:



createdAt  
updatedAt

Timestamp type:

TIMESTAMP

Prefer UTC internally.

The frontend can display times in:

IST

for the HackNEX audience.

---

## 65. Database Time Zone

Recommended:

Store timestamps in UTC.

Example:

Database:

2026-09-08 04:30 UTC

Display:

2026-09-08 10:00 IST

This avoids timezone inconsistencies.

---

## 66. Prisma Schema

Below is the proposed Prisma schema structure.

```
enum FoodPreference {  
  VEG  
  NON_VEG  
}
```

```
enum TeamStatus {  
  DRAFT  
  READY_FOR_PAYMENT
```

```
PAYMENT_PENDING
PAYMENT_VERIFIED
CONFIRMED
CANCELLED
}
```

```
enum RegistrationStatus {
  DRAFT
  READY_FOR_PAYMENT
  PAYMENT_PENDING
  PAYMENT_VERIFIED
  CONFIRMED
  CANCELLED
}
```

```
enum PaymentStatus {
  PENDING
  INITIATED
  PROCESSING
  SUCCESS
  VERIFIED
  FAILED
  EXPIRED
  CANCELLED
}
```

```
enum MediaCategory {
  HERO
  ABOUT
  VENUE
  SPONSOR
  ORGANIZER
  OTHER
}
```

---

## 67. Prisma — User

```
model User {
  id      String      @id @default(uuid())
  name    String
  email   String      @unique
  emailVerified Boolean @default(false)

  accounts Account[]
  sessions Session[]
}
```

```

teams      Team[]
admin      Admin?
participants Participant[]

createdAt  DateTime  @default(now())
updatedAt  DateTime  @updatedAt
}

```

---

## 68. Prisma — Session

```

model Session {
  id      String  @id @default(uuid())
  userId  String
  token   String  @unique
  expiresAt DateTime

  user    User    @relation(fields: [userId], references: [id], onDelete: Cascade)

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  @@index([userId])
}

```

---

## 69. Prisma — Account

```

model Account {
  id          String  @id @default(uuid())
  userId      String
  accountId   String
  providerId  String

  accessToken  String?
  refreshToken String?
  accessTokenExpiresAt DateTime?
  refreshTokenExpiresAt DateTime?
  scope        String?
  password     String?

  user        User    @relation(fields: [userId], references: [id], onDelete: Cascade)

  createdAt  DateTime @default(now())
  updatedAt  DateTime @updatedAt
}

```

```
@@index([userId])
@@unique([providerId, accountId])
}
```

---

## 70. Prisma — Verification

```
model Verification {
  id      String  @id @default(uuid())
  identifier String
  value    String
  expiresAt DateTime

  createdAt DateTime @default(now())
  updatedAt  DateTime @updatedAt

  @@index([identifier])
}
```

---

## 71. Prisma — Team

```
model Team {
  id          String  @id @default(uuid())
  teamName    String  @unique
  captainUserId String  @unique
  status      TeamStatus @default(DRAFT)

  captain    User      @relation(fields: [captainUserId], references: [id])
  participants Participant[]
  registration Registration?

  createdAt  DateTime @default(now())
  updatedAt  DateTime @updatedAt

  @@index([status])
}
```

---

## 72. Prisma — Participant

```
model Participant {
  id      String  @id @default(uuid())
```

```

teamId    String
userId    String?

name      String
email     String    @unique
phone     String
college   String
department String
yearOfStudy String
linkedinUrl String?
foodPreference FoodPreference
isCaptain Boolean    @default(false)

team      Team    @relation(fields: [teamId], references: [id], onDelete: Cascade)
user      User?   @relation(fields: [userId], references: [id])

createdAt DateTime @default(now())
updatedAt DateTime @updatedAt

@@index([teamId])
@@index([college])
@@index([department])
@@index([phone])
}

```

---

## 73. Why **userId** Is Optional for Participants

The captain has a HackNEX account.

The other three participants do not necessarily need accounts because:

Only the captain registers the team.

Therefore:

Captain  
userId → User

while:

Member 2  
userId → NULL

Member 3

userId → NULL

Member 4

userId → NULL

This exactly matches the registration workflow.

---

## 74. Prisma — Registration

```
model Registration {
  id          String      @id @default(uuid())
  registrationId String    @unique
  teamId      String      @unique

  status      RegistrationStatus @default(DRAFT)
  submittedAt DateTime?
  confirmedAt  DateTime?

  team      Team      @relation(fields: [teamId], references: [id])
  payments  Payment[]

  createdAt  DateTime      @default(now())
  updatedAt  DateTime      @updatedAt

  @@index([status])
}
```

---

## 75. Prisma — Payment

```
model Payment {
  id          String      @id @default(uuid())
  registrationId String

  amount      Int
  currency    String      @default("INR")
  provider    String
  transactionId String?    @unique
  providerReference String?
  status      PaymentStatus @default(PENDING)

  verifiedAt  DateTime?

  registration Registration @relation(
```

```

    fields: [registrationId],
    references: [id]
  )

  createdAt    DateTime    @default(now())
  updatedAt    DateTime    @updatedAt

  @@index([registrationId])
  @@index([status])
}

```

---

## 76. Prisma — Domain

```

model Domain {
  id      String  @id @default(uuid())
  name    String
  slug    String  @unique
  description String
  icon    String?

  displayOrder Int
  isActive Boolean @default(true)

  createdAt DateTime @default(now())
  updatedAt  DateTime @updatedAt

  @@index([displayOrder])
}

```

---

## 77. Prisma — ScheduleItem

```

model ScheduleItem {
  id      String  @id @default(uuid())
  title    String
  description String?
  startTime DateTime
  endTime  DateTime?
  location  String?
  day      String

  displayOrder Int
  isActive Boolean @default(true)
}

```

```
createdAt DateTime @default(now())
updatedAt DateTime @updatedAt

@@index([startTime])
@@index([displayOrder])
}
```

---

## 78. Prisma — Prize

```
model Prize {
  id      String @id @default(uuid())
  title   String
  description String?
  amount  Int?
  position Int?

  displayOrder Int
  isActive Boolean @default(true)

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  @@index([displayOrder])
}
```

---

## 79. Prisma — Sponsor

```
model Sponsor {
  id      String @id @default(uuid())
  name    String
  logoUrl String?
  websiteUrl String?
  tier     String?

  displayOrder Int
  isActive Boolean @default(true)

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  @@index([displayOrder])
}
```

---



## 80. Prisma — FAQ

```
model FAQ {
  id      String  @id @default(uuid())
  question String
  answer  String

  displayOrder Int
  isActive Boolean @default(true)

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  @@index([displayOrder])
}
```

---

## 81. Prisma — Announcement

```
model Announcement {
  id      String  @id @default(uuid())
  title   String
  content String
  publishedAt DateTime?
  isPublished Boolean @default(false)

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  @@index([publishedAt])
}
```

---

## 82. Prisma — Media

```
model Media {
  id      String  @id @default(uuid())
  publicId String  @unique
  secureUrl String
  resourceType String
  category MediaCategory

  title   String?
  altText String?
```

```

displayOrder Int
isActive Boolean @default(true)

createdAt DateTime @default(now())
updatedAt DateTime @updatedAt

@@index([category])
@@index([displayOrder])
}

```

---

## 83. Prisma — Admin

```

model Admin {
  id String @id @default(uuid())
  userId String @unique
  isActive Boolean @default(true)

  user User @relation(fields: [userId], references: [id])
  auditLogs AuditLog[]

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}

```

---

## 84. Prisma — AuditLog

```

model AuditLog {
  id String @id @default(uuid())
  adminId String

  action String
  entityType String
  entityId String?
  metadata Json?

  admin Admin @relation(fields: [adminId], references: [id])

  createdAt DateTime @default(now())

  @@index([adminId])
  @@index([entityType])
  @@index([createdAt])
}

```

---

## 85. Full Prisma Schema Structure

The final `schema.prisma` will therefore contain:

### Enums

- FoodPreference
- TeamStatus
- RegistrationStatus
- PaymentStatus
- MediaCategory

### Models

- User
- Session
- Account
- Verification
- Team
- Participant
- Registration
- Payment
- Domain
- ScheduleItem
- Prize
- Sponsor
- FAQ
- Announcement
- Media
- Admin
- AuditLog

---

## 86. Important Prisma/Authentication Note

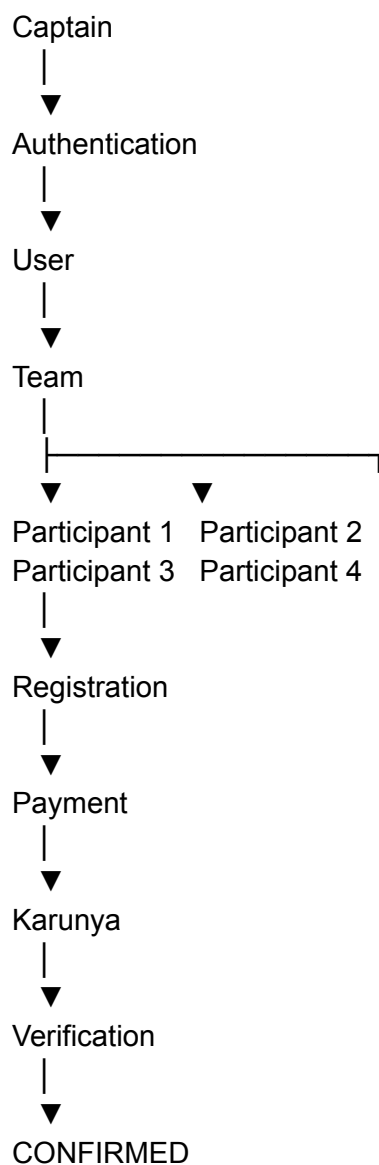
The exact authentication tables and field names should ultimately match the **specific Better Auth adapter/schema version installed in the project**.

Therefore, when implementing authentication, we should generate/update the authentication portion from the installed Better Auth configuration rather than blindly copying a static schema.

The application-specific tables above remain the architectural source of truth.

---

## 87. Registration Data Flow

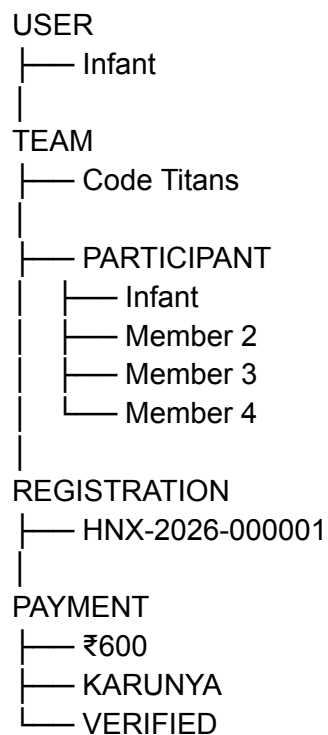


## 88. Complete Registration Database Example

Suppose:

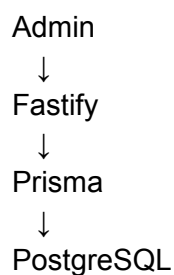
Team Name:  
Code Titans

Database conceptually becomes:



## 89. Admin Query Architecture

For the admin dashboard:



Example:

GET /api/admin/participants?page=1&limit=25

Database:

LIMIT 25

OFFSET 0

This prevents the dashboard from loading all participants simultaneously.

---

## 90. Participant Filtering

The database supports:

College

Department

Year

Food Preference

Team

Registration Status

Example:

GET /api/admin/participants  
?college=KITS  
&foodPreference=VEG  
&page=1  
&limit=25

---

## 91. Payment Reporting

Admins should be able to query:

Total Teams

Confirmed Teams

Pending Payments

Verified Payments

Failed Payments

Total Revenue

Example:

375 teams × ₹600

=

₹225,000

The dashboard should calculate financial totals from verified payment records rather than trusting client-side values.

---

## 92. Data Integrity Rules

The backend must enforce:

1. Team name must be unique.
  2. Participant email must be unique.
  3. A team cannot be confirmed with fewer than 4 participants.
  4. A team cannot be confirmed with more than 4 participants.
  5. Exactly one captain per confirmed team.
  6. Captain must have a valid User account.
  7. Registration must belong to one team.
  8. Payment must belong to one registration.
  9. Payment transaction IDs must be unique.
  10. Confirmation requires verified payment.
  11. Registration after deadline must be rejected.
  12. Cancelled registrations cannot become confirmed without an explicit recovery process.
- 

## 93. Critical Business Rules Matrix

| Rule                        | Enforcement            |
|-----------------------------|------------------------|
| Team exactly 4 members      | Backend transaction    |
| Unique team name            | Database + backend     |
| Unique participant email    | Database               |
| One captain                 | Backend + relationship |
| Captain must authenticate   | Backend                |
| Email verification required | Auth/backend           |
| ₹600 fee                    | Backend                |
| Payment verification        | Backend                |
| Confirmation after payment  | Transaction            |

|                       |                       |
|-----------------------|-----------------------|
| Registration deadline | Backend               |
| Admin access          | Backend authorization |
| Image ownership       | Cloudinary            |
| Content activation    | Database              |

---

## 94. Database Backup

Because registration information is critical:

Neon PostgreSQL



Automated backups



Recovery

The production backup and retention configuration must be verified before opening registration.

---

## 95. Migration Strategy

Prisma migrations should be used.

Development:

```
npx prisma migrate dev
```

Production:

```
npx prisma migrate deploy
```

Database schema changes must go through migrations rather than manually modifying production tables.

---

## 96. Seed Data

The project should include a seed mechanism.



Example:

```
prisma/
├── schema.prisma
├── migrations/
└── seed.ts
```

Seed data can include:

- Initial domains
- Sample schedule
- Sample prizes
- Sample FAQs
- Sample sponsors
- Development admin accounts

Production participant data must never be included in seed scripts.

---

## 97. Development Database

Recommended:

Development



Separate Neon database

Production:

Production



Separate Neon database

Never use the production database during normal development.

---

## 98. Database Environment Variables

`DATABASE_URL="postgresql://..."`

`DIRECT_URL="postgresql://..."`

The exact configuration depends on the Neon/Prisma connection strategy used.

These values must never be committed to GitHub.

---

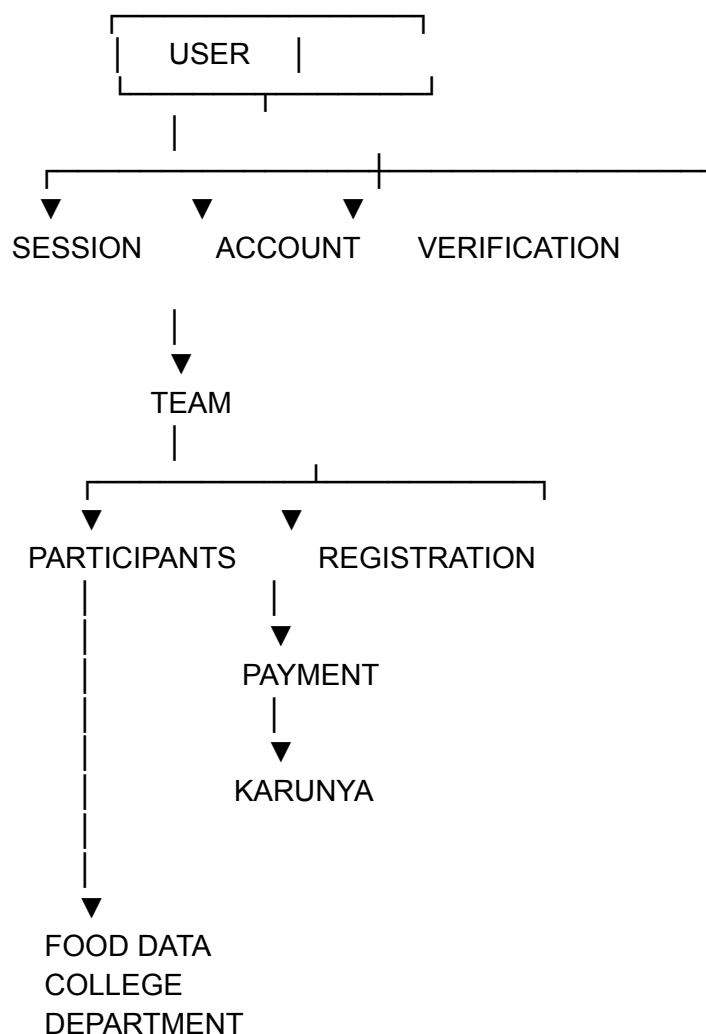
## 99. Database Security

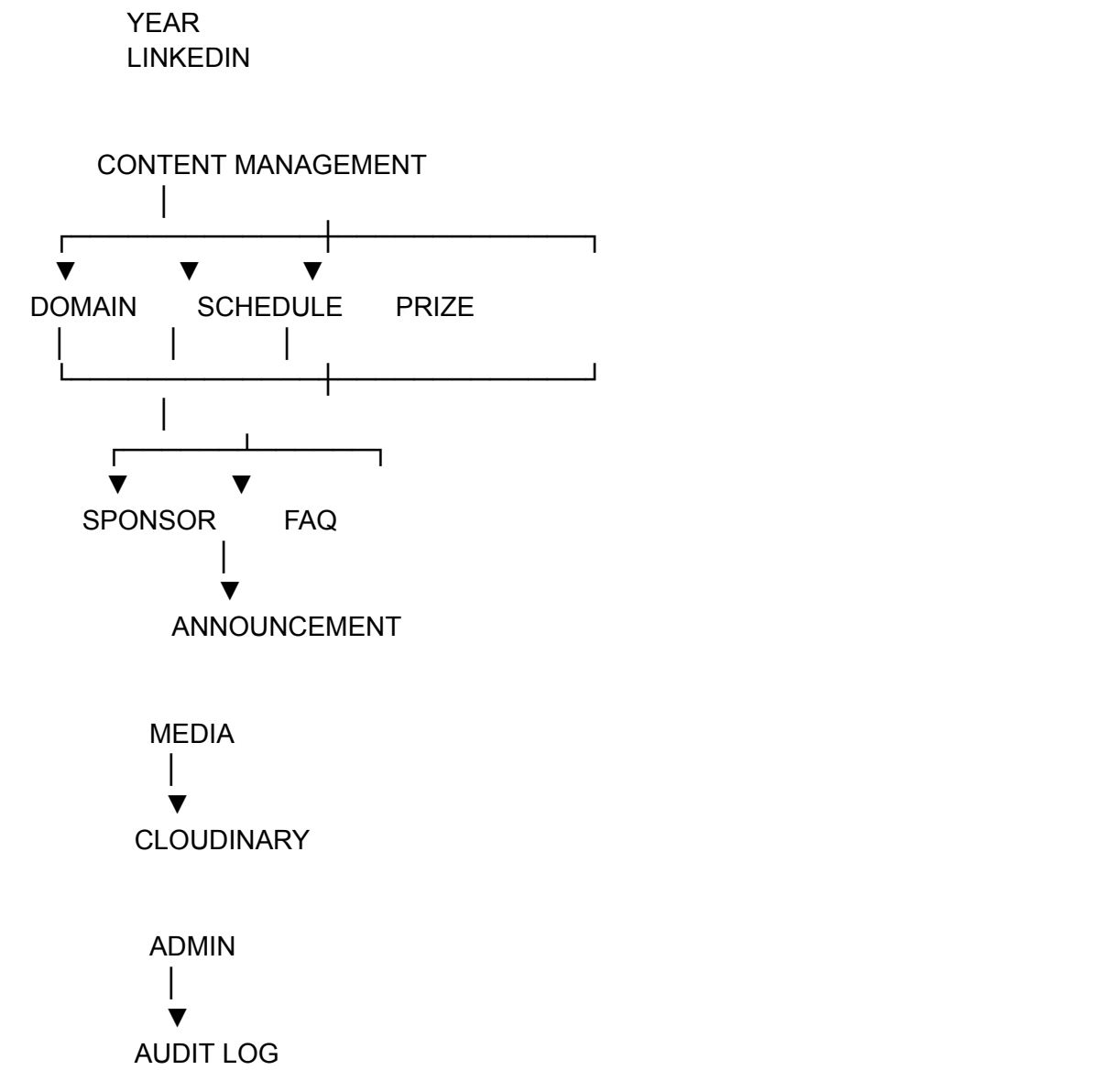
Production database credentials must:

- Remain server-side.
  - Never reach React.
  - Never appear in Git.
  - Never appear in client-side JavaScript.
  - Never appear in public logs.
- 

## 100. Final Database Architecture

The finalized HackNEX database is:





# 101. Final Database Stack

| HACKNEX DATABASE |                         |  |
|------------------|-------------------------|--|
| Database         | PostgreSQL              |  |
| Hosting          | Neon                    |  |
| ORM              | Prisma                  |  |
| IDs              | UUID                    |  |
| Transactions     | PostgreSQL Transactions |  |
| Validation       | Backend + Database      |  |
| Backups          | Neon Managed Backups    |  |

# 102. Final Schema Summary

## 17 core models/entities:

1. User
2. Session
3. Account
4. Verification
5. Team
6. Participant
7. Registration
8. Payment
9. Domain
10. ScheduleItem
11. Prize
12. Sponsor
13. FAQ
14. Announcement
15. Media
16. Admin
17. AuditLog

## 5 enums:

1. FoodPreference
2. TeamStatus
3. RegistrationStatus
4. PaymentStatus
5. MediaCategory